

Waybionic Ground Station — Project Overview

Written for someone new to ROS. This document explains what every folder and file does, and what you need to learn to contribute.

What Is This Project?

This is a **ROS 2 software stack** for controlling the **Annin Robotics AR4** — a 6-axis robotic arm. It supports three hardware versions (MK1, MK2, MK3) and lets you:

- Control the real physical arm
- Simulate the arm in Gazebo (a 3D physics simulator)
- Plan collision-free movements using MoveIt 2
- Visualize everything in RViz

The software runs on **Ubuntu 24.04 with ROS 2 Jazzy** (not Windows — hence the `rosdep` error you saw).

Top-Level Files

File / Folder	What It Does
<code>README.md</code>	Main documentation — start here for installation and usage instructions
<code>LICENSE</code>	MIT open-source license
<code>.gitignore</code>	Tells Git which files to ignore (build outputs, logs, etc.)
<code>.dockerignore</code>	Tells Docker which files to exclude when building a container image
<code>.vscode/</code>	VS Code editor settings — IntelliSense paths for ROS/C++, formatting rules
<code>docker/</code>	Files for running the project inside a Docker container
<code>docs/</code>	Extra documentation: wiring diagrams, schematics, hardware guides

Package Folders

ROS projects are organized into **packages** — self-contained modules that each do one job. This project has 5 packages:

`annin_ar4_description/` — Robot Model

Defines **what the robot looks like and how it moves** — its shape, joints, links, and physical limits. Nothing here runs code; it's pure description files.

File / Folder	What It Does
<code>package.xml</code>	ROS package metadata (name, version, dependencies)
<code>CMakeLists.txt</code>	Tells the build system what files to install
<code>urdf/ar.urdf.xacro</code>	Main robot description — defines the 6 joints and links of the arm
<code>urdf/ar_macro.xacro</code>	Reusable macro for the arm's kinematic structure
<code>urdf/ar_gazebo.urdf.xacro</code>	Adds Gazebo physics properties (friction, inertia, contacts)
<code>urdf/ar_ros2_control.xacro</code>	Tells <code>ros2_control</code> which joints to control and their limits
<code>urdf/ar_gripper_macro.xacro</code>	Defines the gripper's two finger joints
<code>urdf/ar_gripper_ros2_control.xacro</code>	Tells <code>ros2_control</code> how to control the gripper

File / Folder	What It Does
config/mk1.yaml	Robot geometry parameters specific to the MK1 hardware version
config/mk2.yaml	Robot geometry parameters specific to the MK2 hardware version
config/mk3.yaml	Robot geometry parameters specific to the MK3 hardware version
meshes/ar4/	3D model files (.STL) for the arm links — used for visualization
meshes/ar4_gripper/	3D model files for the gripper
meshes/ar4_mk3/	3D model files specific to the MK3 arm

Key concept: URDF (Unified Robot Description Format) and Xacro (a macro language for URDF) describe a robot's physical structure — joints, links, masses, visual shapes — in XML.

annin_ar4_driver/ — Hardware Interface

The **bridge between ROS and the real robot hardware**. It communicates with the microcontrollers (Teensy and Arduino Nano) over USB serial to send commands and read sensor data.

File / Folder	What It Does
package.xml	ROS package metadata
CMakeLists.txt	Build config — compiles 6 C++ source files into a shared library
hardware_interface_plugin.xml	Registers the hardware interface as a pluginlib plugin so ROS can load it
config/driver.yaml	Main driver settings (calibration sequence, velocity control on/off)
config/controllers.yaml	Configures the joint trajectory controller and gripper controller
config/gripper_driver.yaml	Gripper-specific settings (PID gains, current limits)
config/joint_offsets/mk1.yaml	Calibration offsets for MK1 — corrects for physical zero-position differences
config/joint_offsets/mk2.yaml	Calibration offsets for MK2
config/joint_offsets/mk3.yaml	Calibration offsets for MK3
include/ar_hardware_interface.hpp	C++ header: declares the main hardware interface class
include/teensy_driver.hpp	C++ header: declares the Teensy serial communication driver
include/arduino_nano_driver.hpp	C++ header: declares the Arduino Nano gripper driver
include/ar_servo_gripper_hw_interface.hpp	C++ header: declares the gripper hardware interface
include/gripper_overcurrent_protection.hpp	C++ header: declares over-current protection logic
src/ar_hardware_interface.cpp	Implements read/write loops — reads joint positions, sends joint commands
src/teensy_driver.cpp	Implements serial protocol with Teensy — calibration, E-stop, joint updates
src/arduino_nano_driver.cpp	Implements serial communication with the Arduino Nano gripper
src/ar_servo_gripper_hw_interface.cpp	Implements gripper controller management
src/gripper_overcurrent_protection.cpp	Implements current-limiting algorithm with hysteresis
src/run_arduino_nano.cpp	Standalone executable for testing the gripper in isolation

<code>launch/driver.launch.py</code> File / Folder	Python launch file — starts all driver nodes, loads URDF, spawns controllers What It Does
<code>scripts/reset_estop.sh</code>	Shell script to reset the emergency stop
<code>urdf/ar.urdf.xacro</code>	Alternative URDF used specifically by the driver package

annin_ar4_firmware/ — Microcontroller Code

Code that runs **directly on the microcontrollers** (not on your PC). These are Arduino sketches (.ino files).

File	What It Does
<code>AR4_tensy/AR4_tensy.ino</code>	Firmware for the Teensy 3.6 microcontroller — controls 6 stepper motors, reads encoders and limit switches, handles calibration, communicates with the PC over serial
<code>AR4_nano/AR4_nano.ino</code>	Firmware for the Arduino Nano — controls the servo gripper via PWM signals, monitors motor current

Key concept: The Teensy and Arduino are separate computers that plug into the PC via USB. They handle low-level motor control in real-time. The ROS driver talks to them over serial (like a text conversation over USB).

annin_ar4_gazebo/ — Simulation

Sets up a **Gazebo simulation** so you can test the arm in a virtual 3D world without needing real hardware.

File	What It Does
<code>package.xml</code>	ROS package metadata
<code>CMakeLists.txt</code>	Installs launch and world files
<code>launch/gazebo.launch.py</code>	Starts Gazebo, spawns the robot model, bridges clock signals, starts controllers
<code>worlds/empty.world</code>	A simple empty Gazebo world (just a ground plane and physics engine)

annin_ar4_moveit_config/ — Motion Planning

Configures **Movelit 2** — the motion planning framework that calculates smooth, collision-free paths for the arm.

File / Folder	What It Does
<code>package.xml</code>	ROS package metadata
<code>CMakeLists.txt</code>	Installs all config, launch, rviz, srdf, urdf directories
<code>launch/demo.launch.py</code>	Launches RViz + MoveIt in demo mode — no real hardware needed
<code>launch/moveit.launch.py</code>	Full launch: move_group node, RViz, optional servo node
<code>config/controllers.yaml</code>	Maps MoveIt to the hardware controllers
<code>config/joint_limits.yaml</code>	Maximum velocity and acceleration for each joint
<code>config/kinematics.yaml</code>	Configures the KDL inverse kinematics solver
<code>config/moveit_servo.yaml</code>	Real-time servo control parameters (for jogging the arm live)
<code>config/ompl_planning.yaml</code>	Configures OMPL planners (RRT, RRTstar, PRM) for path planning
<code>config/pilz_planning.yaml</code>	Configures Pilz safe motion planner (for linear/circular movements)
<code>srdf/ar.srdf.xacro</code>	SRDF — defines planning groups, end-effector links, collision exclusions

File / Folder	What It Does
srdf/ar_macro.srdf.xacro	SRDF macro for the arm planning group
srdf/ar_gripper_macro.srdf.xacro	SRDF macro for the gripper planning group
rviz/moveit.rviz	RViz configuration for the MoveIt interface
rviz/moveit_with_camera.rviz	RViz config that also shows camera sensor data
urdf/fake_ar.urdf.xacro	Fake/simulated URDF used in demo mode (no real hardware)

docker/ — Containerization

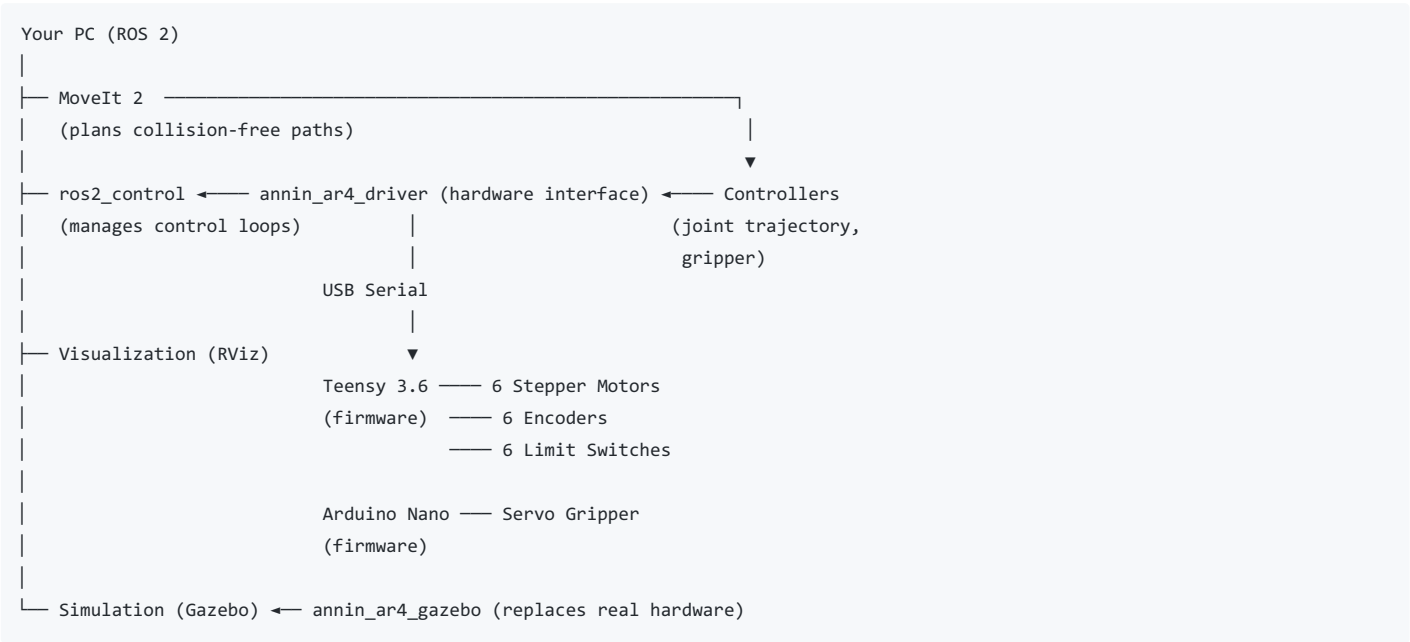
Lets you run the entire ROS environment inside a **Docker container** — useful since ROS runs on Linux and you're on Windows.

File	What It Does
Dockerfile	Instructions to build a Docker image: starts from ROS Jazzy base, installs dependencies, builds the project
entrypoint.sh	Script that runs when the container starts — sources the ROS setup file so all commands work

docs/ — Documentation

File	What It Does
ar4_dip_switch_settings.png	Image showing how to configure the DIP switches on the MK3 control board
gripper_overcurrent_protection.md	Guide for adding a current sensor (ACS712) to the gripper to prevent stalling
gripper_overcurrent_protection_schematic.png	Electrical wiring diagram for the current sensor circuit

How Everything Connects



What You Need to Learn

Since you're new to ROS, here is a prioritized learning roadmap:

1. Linux / Ubuntu (Essential — Week 1)

ROS 2 runs on Linux. You need basic command-line skills.

- Navigation: `cd`, `ls`, `pwd`, `mkdir`, `cat`
- File editing: `nano` or `vim`
- Permissions: `chmod`, `sudo`
- Processes: `ps`, `kill`, `Ctrl+C`
- **Why:** Almost all ROS commands are run in a Linux terminal. Without this, nothing else works.

2. Python (Essential — Weeks 1–3)

All ROS 2 launch files are written in Python. You'll also write nodes in Python.

- Variables, functions, classes
- Importing modules
- Basic file I/O
- **Why:** Launch files (`.launch.py`) are Python scripts. Most beginner ROS tutorials use Python nodes.

3. ROS 2 Core Concepts (Essential — Weeks 2–6)

This is the main thing to learn.

- **Nodes** — individual programs that do one job
- **Topics** — message streams nodes publish/subscribe to
- **Services** — request/response communication between nodes
- **Actions** — long-running tasks with feedback (used by MoveIt)
- **Parameters** — configurable values for nodes
- **Launch files** — Python scripts that start multiple nodes at once
- **colcon** — the ROS 2 build tool
- **ros2 CLI** — commands like `ros2 topic list`, `ros2 node info`, `ros2 run`
- **Resource:** [ROS 2 Jazzy tutorials](#) — do these in order

4. URDF / Xacro (Important — Weeks 4–6)

Describes a robot's physical structure in XML.

- Links and joints
- Transforms (TF)
- Xacro macros
- **Why:** The `annin_ar4_description` package is entirely URDF/Xacro. You need this to understand or modify the robot model.

5. C++ Basics (Important — Weeks 4–8)

The hardware driver (`annin_ar4_driver`) is written in C++.

- Classes, headers, source files
- Pointers and references
- Build systems (CMake basics)
- **Why:** If you ever need to modify the hardware interface or driver, it's all in C++.

6. MoveIt 2 (Advanced — Months 2–3)

The motion planning framework.

- Planning groups and end effectors
- `move_group` node
- Python/C++ MoveIt API
- SRDF concepts
- **Resource:** [MoveIt 2 documentation](#)
- **Why:** This is how you program the arm to move to specific positions autonomously.

7. Gazebo (Advanced — Months 2–3)

3D physics simulation.

- World files and SDF format
- Spawning robots in simulation
- Gazebo-ROS bridge (`ros_gz_bridge`)
- **Why:** Lets you test code safely without risking damage to the real arm.

8. Docker (Useful — Weeks 2–4)

Containerization — packages the entire ROS environment into a portable box.

- `docker build` , `docker run` , `docker exec`
- Volumes and device mounting
- `rocker` (ROS-specific Docker tool for GUI support)
- **Why:** Since you're on Windows, Docker + WSL is likely your path to running this project at all.

9. Arduino / Embedded C (Optional — Later)

For modifying the microcontroller firmware.

- Arduino IDE
- `AccelStepper` library
- Serial communication
- PWM signals
- **Why:** Only needed if you need to change motor control behavior at the firmware level.

Recommended First Steps

1. Install **WSL 2** with Ubuntu 24.04 on your Windows machine
2. Inside WSL, install **ROS 2 Jazzy** following the official guide
3. Complete the official **ROS 2 beginner tutorials** (nodes, topics, services)
4. Clone this repo inside WSL and run `colcon build`
5. Try `ros2 launch annin_ar4_moveit_config demo.launch.py` to see the robot in RViz without any hardware

Created by Richard — personal notes folder